

Base de conocimiento vectorial — Documento técnico

CODEFEST AD ASTRA 2026 · Etapa 1 · Decisiones de diseño de la base vectorial, el grafo de conocimiento y el módulo de recuperación

1. Qué se entrega

La base cubre **89.064** fragmentos procedentes de **1.715** documentos (1.648 archivos distintos del corpus de ADL) repartidos entre los tres fenómenos del reto. Se construyó con **dos encoders complementarios**, cada uno con su propio índice FAISS (Sección 4.4), y se acompaña del componente bonus: un grafo de conocimiento de 226.821 entidades y 5.118.081 relaciones que cubre las dos particiones.

Índice	Encoder	Dim.	Fragmentos	Docs.	Partición del corpus
encoder_multilingual-e5-base	intfloat/multilingual-e5-base	768	79.178	1.708	PDF de observatorios (757 docs) y artículos web en JSON (951 docs)
encoder_bge-m3	BAAI/bge-m3	1024	9.886	7	Datasets XLSX del AI Index (8901 filas) y mapas .pbf (985 municipios)

El reparto no es arbitrario: el corpus se dividió por tipo de fuente para poder procesarlo en paralelo, y cada mitad se codificó con el encoder que mejor le convenía. La consecuencia es que un fragmento vive en uno solo de los dos espacios vectoriales, y eso condiciona cómo se combinan en la recuperación (Sección 5 de este informe).

2. Preprocesamiento y extracción de texto

Cada formato del corpus exige una vía de extracción distinta (Sección 2.1). En todos los casos el texto resultante se normaliza a UTF-8, se colapsan espacios y caracteres de control, y se eliminan cabeceras, pies y numeración de página repetidos.

Formato	Extracción	Unidad de fragmentación
PDF	Texto por página respetando el orden de lectura; se descartan figuras y elementos decorativos sin contenido legible.	Ventana deslizante sobre el texto continuo del documento.
JSON	Se interpreta el objeto y se concatenan solo los campos de contenido (título y cuerpo). Los campos descriptivos (url, fecha, autores) se guardan como metadata, no se mezclan con el texto.	Bloques del artículo.
XLSX	Se lee la cabecera y luego cada registro como pares <i>columna: valor</i> , de modo que cada valor conserva el nombre de su columna como contexto.	Una fila = un fragmento.
PBF	Se decodifican las teselas vectoriales, se recorren las capas y se leen los atributos de cada elemento del mapa. Como el mismo municipio se repite en varios niveles de zoom, se deduplica: de 8.522 apariciones quedan 985 municipios únicos.	Un municipio = un fragmento, redactado en prosa.

3. Estrategia de chunking y su justificación

No se usó una única estrategia para todo el corpus, porque los cuatro tipos de fuente tienen estructuras que no se parecen en nada. Lo que sí es común es el criterio: el fragmento debe ser la unidad más pequeña que todavía responde por sí sola, sin depender de lo que había antes.

- **PDF — ventana deslizante de 220 palabras con 40 de solapamiento.** Los informes de observatorios son texto corrido de decenas de páginas, sin marcado estructural fiable tras la extracción. Una ventana de 220 palabras (media real: 216) cabe holgadamente en el límite de 512 tokens del encoder y deja margen para el prefijo. El solapamiento de 40 palabras es lo que evita que una idea que cae justo en la frontera entre dos ventanas se pierda para ambas. Cada fragmento guarda además la página de inicio y de fin, lo que permite rastrear cualquier resultado hasta el PDF original.
- **JSON — bloques del artículo.** Los artículos web ya vienen con el cuerpo separado en párrafos, así que se respeta esa segmentación del autor, que es la más cercana a la estructura semántica del texto.

- **XLSX — una fila por fragmento.** En un dataset tabular la fila es la unidad semántica completa: partirla no aporta nada y juntar varias mezcla registros que no tienen relación. Cada celda se serializa con el nombre de su columna delante para que el encoder sepa qué significa cada valor.
- **PBF — un municipio por fragmento.** Los mapas no son texto, así que el módulo de mapas convierte los atributos de cada municipio (economías ilícitas, grupos armados presentes, indicadores) en un párrafo en lenguaje natural. El resultado se comporta como cualquier otro fragmento del corpus.

Complejidad lingüística. El requisito de la Sección 3.3 se aplica en dos puntos. En el índice, las particiones de JSON, XLSX y PBF cortan siempre en límites naturales (párrafo, fila, entidad del mapa). En la entrega, *generador.py* vuelve a imponerlo sobre el texto que efectivamente se reporta: recorta los bordes de cada fragmento hasta la oración completa más cercana y, si un fragmento supera las 250 palabras, lo divide en ventanas que cortan solo entre oraciones (Sección 9.2.1).

4. Encoders y criterios de selección

El corpus mezcla español, inglés y portugués, y las consultas de evaluación llegan en los tres idiomas, así que el soporte multilingüe nativo era la condición eliminatoria: sin él, una pregunta en español no puede recuperar un informe en inglés. Los dos modelos elegidos son **encoders** de la familia BERT/XLM-R, con licencia MIT, disponibles públicamente en HuggingFace.

Criterio	intfloat/multilingual-e5-base	BAAI/bge-m3
Multilingüe	100 idiomas; entrenado con pares consulta-pasaje en varios idiomas, que es justo la tarea del reto.	100+ idiomas, muy sólido en recuperación translingüe.
Dimensión	768 — suficiente para el tamaño del corpus y cuatro veces más barato de almacenar y comparar que un modelo grande.	1024, con mayor capacidad expresiva.
Longitud máx.	512 tokens, que es lo que fija el tamaño de chunk.	8.192 tokens, holgado para filas y descripciones de mapas.
Rendimiento	Fuerte en la pista multilingüe de MTEB/BEIR en recuperación densa.	Referencia actual en recuperación multilingüe.
Coste	278M parámetros: permite indexar 79.178 fragmentos en tiempo razonable.	568M parámetros: se reservó para la partición pequeña.

Convención de codificación. e5 fue entrenado con prefijos de rol y omitirlos degrada la recuperación: los fragmentos se indexaron con «*passage:* » y las consultas se codifican con «*query:* ». bge-m3 no usa prefijo. Ambas convenciones se verificaron reconstruyendo vectores del índice y comparándolos con una recodificación del mismo texto: la similitud es exactamente 1,0 con el prefijo correcto y baja a 0,94 con el equivocado.

5. Índice FAISS

Los dos índices son **IndexFlatIP con vectores normalizados a norma unitaria**, serializados con *faiss.write_index()* y cargables con *faiss.read_index()* sin dependencias adicionales. Con norma unitaria el producto interno es exactamente la similitud coseno (Sección 8.2), de modo que el índice devuelve directamente la métrica que pide el reto.

Se descartaron los índices aproximados (IVF, HNSW) a propósito. Con 89.064 vectores, la búsqueda exacta tarda del orden de 10 ms por consulta: no hay ningún problema de latencia que resolver, y a cambio un índice aproximado introduciría una pérdida de exactitud que se pagaría directamente en NDCG. La regla es simple: no se sacrifica recall para ganar una velocidad que no hace falta.

El almacén de metadata es un *metadata.jsonl* por índice, con un objeto por línea y **la línea i correspondiendo al vector i** del índice FAISS (Sección 5.3). Cada objeto trae los ocho campos obligatorios de la Tabla 1 más la ruta original y, en los PDF, las páginas de origen.

6. Grafo de conocimiento (componente bonus)

El grafo se construyó sobre los mismos fragmentos indexados, en tres etapas (Sección 7.2):

- **Reconocimiento de entidades.** Se usó *GLiNER* (urchade/gliner_multi-v2.1), un modelo NER multilingüe *zero-shot*: en vez de una taxonomía fija, se le pasa como texto la lista de tipos que interesan al reto —país, organización, grupo armado, institución del Estado, tecnología, sistema de armas, satélite, capacidad contraespacial, órbita, instrumento legal, economía ilícita, recurso natural, infraestructura crítica, evento o conflicto, entre otros—. Umbral de confianza 0,5 sobre el texto de los chunks.
- **Extracción de relaciones.** Las relaciones se derivan de la coocurrencia de entidades dentro de un mismo fragmento, que es la unidad en la que hay evidencia textual verificable de que las dos entidades tienen algo que ver.
- **Integración.** Cada nodo guarda los *chunk_id* y *doc_id* donde aparece la entidad, y cada arista guarda los *chunk_id* que la sostienen. Esa es la vinculación con la base vectorial (Sección 7.3): del grafo se puede volver siempre al texto que respalda cada afirmación. Se exporta como *grafo.graphml* desde NetworkX.

El resultado son **226.821 entidades** y **5.118.081 relaciones**, unos 843.672 vínculos entidad–fragmento. El archivo pesa cerca de 1 GB, lo que descarta cargarlo con *networkx.read_graphml* en cada consulta: *generador.py* lo recorre con un parser incremental **una sola vez para las 50 consultas** (45 segundos, memoria acotada), aprovechando que GraphML emite primero todos los nodos y después todas las aristas.

7. Módulo de recuperación

7.1 El problema de combinar dos espacios disjuntos

Las estrategias clásicas de fusión (CombSUM, CombMNZ, RRF) suponen que los dos índices ordenan *el mismo conjunto* de fragmentos y que discrepan en el orden. Aquí no es el caso: las particiones son disjuntas, así que un fragmento aparece en un ranking y en el otro no. Aplicar RRF sin más pondría el mejor resultado de la partición tabular al mismo nivel que el mejor de la partición de informes, por irrelevante que fuera, solo porque ambos son «el primero de su lista». Y sumar cosenos crudos tampoco sirve, porque cada encoder tiene su propia escala de similitud.

La solución adoptada es usar los dos índices como fuentes de *recall* y **reproyectar todos los candidatos a un único espacio común** antes de ordenarlos. Los fragmentos que ya están en el índice de e5 aportan su vector almacenado (exacto, sin recodificar); los que vienen de bge-m3 se codifican con e5 sobre su texto. Así todos se puntúan con la misma vara, sin ningún modelo generativo de por medio.

7.2 Flujo completo

- **Recuperación.** Top-200 del índice e5 (consulta codificada con «*query*: ») + top-100 del índice bge-m3 + hasta 100 fragmentos propuestos por el grafo. La unión forma el pool de candidatos.
- **Calibración.** Coseno de cada candidato contra la consulta en el espacio común, estandarizado sobre el pool (media 0, desviación 1). Trabajar en unidades de desviación típica hace que el peso del grafo signifique lo mismo en una consulta donde los cosenos están muy juntos que en otra donde están separados.
- **Refinamiento.** De los 60 mejores se recorta el texto a oraciones completas y, si supera las 250 palabras, se elige la ventana que más se parece a la consulta. Se puntúa la ventana que se va a entregar, no el chunk entero: es exactamente lo que evalúa NDCG@10, que juzga la relevancia sobre el campo *text*.
- **Fusión con el grafo** (Sección 8.5, punto 4) y ordenación final.

$$\text{puntuación}(c) = z(\text{coseno}(c)) + 0,5 \cdot \text{grafo}(c)$$

- **Agregación a documento** (Sección 8.6): max-pooling más una bonificación de 0,3 por cada fragmento adicional del mismo documento, de modo que un documento con varios fragmentos relevantes gane a uno con un único acierto aislado. La agrupación se hace por campo *fuentes* y no por *doc_id*, porque la evaluación empareja documentos por el archivo original (Sección 10.2.1): dos *doc_id* del mismo archivo gastarían dos de los tres cupos en el mismo documento.

7.3 Cómo puntúa el grafo

Las entidades de la consulta se localizan buscando sus n-gramas contra el vocabulario de nodos del grafo —que es el que produjo GLiNER al construirlo, así que ambos lados comparten inventario (Sección 8.5, punto 1)—. Es la forma exacta, y mucho más barata, de lo que hacía la búsqueda por expresión regular nodo a nodo. A partir de ahí se recuperan los fragmentos de esas entidades y de sus vecinos de primer orden:

$$\text{bruto}(c) = \sum w(e) \text{ sobre las entidades } e \text{ de la consulta mencionadas en } c + 0,15 \cdot \sum w(e) \cdot w(v) \text{ sobre los vecinos } v \text{ de primer orden}$$

$$\text{grafo}(c) = \text{bruto}(c) \cdot (1 + 0,4 \cdot (\text{entidades_cubiertas}(c) - 1)) / \sqrt{(\text{entidades_totales}(c))}$$

Los correctivos de esa fórmula salieron de mirar lo que devolvía la versión ingenua —sumar pesos IDF y ordenar—, que era casi inservible:

- **El peso se mide contra el corpus, no contra el grafo.** El vocabulario del grafo contiene sustantivos genéricos que el NER marcó como entidad en un par de fragmentos sueltos («amenazas», «servicios», «pruebas»). Pesarlos por su frecuencia *dentro del grafo* les daba el peso máximo —una detección única parecía máxima especificidad— y arrastraban resultados sin ninguna relación con la pregunta.
- **Una palabra suelta solo cuenta si la consulta la escribe como nombre propio o como sigla** («Colombia», «GAOR», «LEO»). Medir la frecuencia tampoco bastaba: como el corpus es mayoritariamente inglés, cualquier palabra española resulta «rara» y «pruebas» acababa pesando más que «spoofing». La mayúscula del texto original sí separa las entidades que anclan una consulta de las que no. Con esta regla el grafo aporta evidencia en 34 de las 50 consultas, con entidades como «grupos armados organizados residuales» o «corea del norte»; en las 16 restantes no aporta nada, que es preferible a inventar evidencia.
- **El divisor por densidad de entidades** corrige el sesgo que ponía en cabeza a las páginas de bibliografía y a las tablas de datos: no son relevantes para nada, pero mencionan tantas entidades que ganaban por pura acumulación.
- **La expansión hacia vecinos se limita a los 25 mejor respaldados** por entidad, en vez de expandir a ciegas por un grafo con nodos de decenas de miles de aristas.

8. Restricción sobre modelos generativos

Ninguna etapa de la construcción ni de la recuperación usa arquitecturas decoder (Sección 8.3). Los dos modelos de embedding son encoders; GLiNER, usado solo para construir el grafo, también lo es (DeBERTa). No hay reordenamiento por LLM, ni reformulación o expansión generativa de la consulta, ni síntesis de fragmentos: el texto que se entrega es literalmente el del corpus, recortado en límites de oración. Todo el reordenamiento se hace con similitud coseno, pesos IDF y metadata.

9. Reproducibilidad

`python entrega/generador.py` lee `entrega/consultas.txt`, carga los dos índices y el grafo, y reescribe `entrega/resultados.jsonl` con las 50 líneas del formato de la Sección 9. Tarda unos tres minutos en total. Las versiones de las librerías están fijadas en `requirements.txt`, porque un cambio de versión del encoder cambia los vectores y con ellos el ranking. `python modulos/validar_resultados.py` comprueba después el esquema completo: 50 líneas en orden, 3 documentos y 10 fragmentos por consulta, ningún fragmento por encima de 250 palabras y todos los identificadores existentes en la base entregada.